

CH:7 LINEAR DISCRIMINANT FUNCTION-BASED CLASSIFIER

Contents

Linear Discriminant Functions	1
Perceptron	2
Working of Perceptron Algorithm (Step-by-Step).....	3
Artificial Neural Network (ANN)	5
Activation Function	6
Support Vector Machine	7
Types of SVM	8

Linear Discriminant Functions

Linear Discriminant Functions (LDFs) are mathematical tools used in pattern recognition and machine learning to classify data points into different categories. They are the foundation of Linear Discriminant Analysis (LDA) and many linear classifiers like Perceptron and Support Vector Machines (SVMs).

Role in Classification

- **Goal:** To assign input vectors to one of the predefined classes based on a linear combination of their features.

- **Form:** For a 2-class problem, the linear discriminant function is:

$$g(x) = w^T x + w_0$$

- x : input feature vector
- w : weight vector
- w_0 : bias (intercept)

- The sign of $g(x)$ determines class membership:
 - If $g(x) > 0$, classify as **Class 1**
 - If $g(x) < 0$, classify as **Class 2**
- In multi-class classification, multiple discriminant functions $g_1(x), g_2(x), \dots, g_k(x)$ are used, and a point is assigned to the class with the highest score.

Decision Boundaries and Hyperplanes

- The decision boundary is the set of points where the discriminant function equals zero:

$$g(x) = w^T x + w_0$$

- This equation defines a hyperplane in the feature space:
 - In 2D: a line
 - In 3D: a plane
 - In higher dimensions: a hyperplane

- The hyperplane separates the feature space into regions corresponding to different class predictions.
- The orientation of the hyperplane is controlled by the direction of the weight vector w .
- The distance of any point x from the hyperplane can be measured and used to interpret the classifier's confidence.

Linear Separability

- **Linearly separable data:** Two (or more) classes are linearly separable if there exists a hyperplane that perfectly separates the classes without error.
- Mathematically, for all data points x_i , there exists w and w_0 such that:

$$\begin{array}{ll} w^T x_i + w_0 > 0 & \text{for Class 1} \\ w^T x_i + w_0 < 0 & \text{for Class 2} \end{array}$$

- When data is not linearly separable, linear discriminant functions may perform poorly, requiring:
 - Use of non-linear transformations
 - Switching to non-linear classifiers

Perceptron

Perceptron is a supervised learning algorithm used for binary classification problems. It is the simplest type of artificial neural network, where it models a single neuron that learns to classify input data into two classes by adjusting its weights based on the input and the error in prediction.

Objective: To learn a linear boundary (hyperplane) that separates data points of **two classes** using labelled training data.

Perceptron Model: The Perceptron computes a **weighted sum** of the input features and applies an **activation function** (typically the sign function):

$$f(x) = w^T x + b$$

$x = [x_1, x_2, \dots, x_n]$: Input vector
 $w = [w_1, w_2, \dots, w_n]$: Weight vector
 b : Bias term

$f(x)$: Linear combination of inputs and weights

Output Decision Rule:

$$y = \begin{cases} 1 & \text{if } f(x) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Advantages

1. **Simple to implement:** Easy and intuitive algorithm for binary classification.
2. **Fast for linearly separable data:** Converges quickly when data is separable.
3. **Supports online learning:** Can learn incrementally with new data points.
4. **Forms neural network base:** It's the foundation of complex neural models.
5. **Low computation cost:** Lightweight and efficient for small-scale tasks.

Disadvantages

1. **Can't handle non-linear data:** Fails completely for XOR-like patterns.
2. **Only binary classification:** Needs extension for multi-class problems.
3. **No confidence or probability:** Outputs are hard decisions only.
4. **Sensitive to feature scale:** Needs normalization to avoid bias.
5. **No feature abstraction:** Lacks hidden layers to model complex patterns.

Working of Perceptron Algorithm (Step-by-Step)

1. Initialize Weights and Bias

- Start with small random weights (often 0) and bias $b=0$.
- Set a learning rate η (e.g., 0.01 or 1).

2. Input a Training Sample

- Take an input feature vector $x=[x_1, x_2, \dots, x_n]$
- With its label y (either $+1/-1$ or $0/1$, depending on form used).

3. Compute Output/Weighted Sum

$$z = w^T x + b$$

4. Apply Activation Function

- Use a step function (sign function):

$$\hat{y} = \text{sign}(z) \quad \text{or} \quad \hat{y} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

5. Compare Prediction to Actual Label

- If prediction $\hat{y}$ is correct, do nothing.
- If wrong, apply the update rule:

- For label in $\{+1, -1\}$:

$$w = w + \eta y x, \\ b = b + \eta y$$

- For label in $\{0, 1\}$:

$$w = w + \eta (y_{\text{true}} - \hat{y}) x, \\ b = b + \eta (y_{\text{true}} - \hat{y})$$

6. Repeat for All Training Samples

- Continue until all data points are classified correctly or max epochs are reached.

Question
Given: Input $x=[2, 3]$
Initial wt (w) = $[0, 0]$
bias (b) = 0
Learning rate (η) = 1

Calculated updated weight + bias when,
(a) True label $y = +1$
(b) True label $y_{\text{true}} = 1$, predicted $\hat{y} = 0$

Answer

$$(a) \quad z = w^T x + b = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 & 3 \end{bmatrix} = 0$$

$$\text{as } z = 0, \quad \hat{y} = \text{sign}(z) = 0$$

$$\therefore w = w + \eta y x = [0, 0] + 1 \times 1 \times [2, 3] = [2, 3]$$

$$b = b + \eta y = 0 + 1 \times 1 = 1$$

(b) Given $\hat{y} = 0$, (no need to calculate)

$$w = w + \eta (y_{\text{true}} - \hat{y}) x = [0, 0] + 1(1-0)[2, 3] \\ = [2, 3]$$

$$b = b + \eta (y_{\text{true}} - \hat{y}) = 0 + 1(1-0) = 1$$

Question (Perceptron Learning Algorithm)

You are given a binary classification problem with the following training data:

Use the Perceptron Learning Algorithm to find the final weight vector and bias.

Assume the initial weight vector $w = [0, 0]$, bias $b = 0$, and learning rate $\eta = 1$.

Perform epoch-wise updates until convergence.

Use the sign activation function: $\text{sign}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

Sample	x1	x2	Class (Label)
1	2	1	+1
2	-1	-3	-1
3	1	-2	-1
4	2	2	+1

Answer

Perceptron Update Rule,

$$\text{If } \hat{y} \neq y, \quad \begin{cases} w = w + \eta y x \\ b = b + \eta y \end{cases}$$

Epoch 1 $w = [0, 0], b = 0$

x	y	
[2, 1]	+1	$z = w^T x + b = 0$ $\hat{y} = \text{sign}(z) = +1$ } $\hat{y} = y$ $\therefore$ no update.
[-1, -3]	-1	$z = 0$ $\hat{y} = +1$ } $\hat{y} \neq y$ need update. $\therefore w = [0, 0] + 1 \cdot (-1) [-1, -3] = [1, 3]$ $b = 0 + 1 \cdot (-1) = -1$
[1, -2]	-1	$z = [1, 3] [1, -2] - 1 = 1 - 6 - 1 = -6$ $\hat{y} = -1$ ($\hat{y} = y$) $\therefore$ no update.
[2, 2]	+1	$z = [1, 3] [2, 2] - 1 = 2 + 6 - 1 = 7$ $\hat{y} = +1$ ($\hat{y} = y$) $\therefore$ no update.

$\therefore w = [1, 3] \quad b = -1$

As 3 samples have converged not all, we have to again repeat the steps.

Epoch 2

x	y	
[2, 1]	+1	$z = [1, 3] [2, 1] - 1 = 2 + 3 - 1 = 4$ $\hat{y} = +1$ $\therefore$ no update.
[-1, -3]	-1	$z = [1, 3] [-1, -3] - 1 = -1 - 9 - 1 = -11$ $\hat{y} = -1$ $\therefore$ no update

$[1, -2]$	-1	$z = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 1, -2 \end{bmatrix} - 1 = 1 - 6 - 1 = -6$ $\hat{y} = -1 \therefore \text{no update.}$
$[3, 2]$	+1	$z = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 3, 2 \end{bmatrix} - 1 = 2 + 6 - 1 = 7$ $\hat{y} = +1 \text{ no update.}$

$\therefore \text{At } w = \begin{bmatrix} 1, 3 \end{bmatrix} + b = -1$

All samples have converged,

This is the decision boundary learned by the Perceptron algorithm,

$$w_1 x_1 + w_2 x_2 + b \geq 0$$

$$\therefore 1x_1 + 3x_2 - 1 = 0$$

$$\Rightarrow x_1 + 3x_2 = 1$$

This is a straight line equation, which divides the plane in 2 halves —

→ point above it = classified as +1

→ " below = " " -1

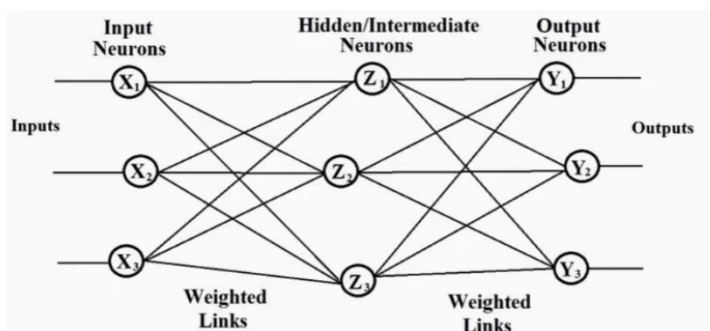
Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of biological neurons in the brain. It is used to learn patterns from data and make predictions or classifications. It learns from data, adapts, and is useful for both linear and non-linear pattern recognition tasks. **Basic Structure of ANN**

1. Input Layer:

- Takes the raw features (e.g., height, weight).
- Each node represents one feature.

2. Hidden Layer(s):



- Processes inputs with **weights and activation functions**.
- Can be one or many layers deep.
- Learns intermediate representations or features.

3. Output Layer:

- Produces the final result (e.g., class label or regression value).
- Number of nodes = number of output classes (for classification).

How ANN Works (Forward Propagation):

1. Each input x_i is multiplied with a weight w_i .
2. All weighted inputs are summed and added with a bias term b .
3. The result goes through an activation function f :

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + \dots + w_nx_n + b \\ a &= f(z) \end{aligned}$$

4. This output is passed to the next layer and the process repeats.

Learning in ANN (Training):

- Uses **backpropagation** and **gradient descent** to minimize **error/loss**.
- Weights are updated to reduce the gap between predicted and actual output.

Key Components:

- **Weights (w):** Controls the strength of connections.
- **Bias (b):** Shifts activation, improves flexibility.
- **Activation function (f):** Introduces non-linearity.
- **Loss function:** Measures prediction error.
- **Optimizer (e.g., Gradient Descent):** Updates weights during training.

Applications in Pattern Recognition:

- Handwriting recognition
- Face or object detection
- Speech recognition
- Medical diagnosis (based on symptoms/patterns)
- Fraud detection

Activation Function

An **activation function** is a mathematical function used to **introduce non-linearity** in the output of a computational unit (like a neuron in a model). It is applied to the output of a model's internal computation (typically after calculating a weighted sum of inputs) to determine the final output signal.

Why is it important?

- It **introduces non-linearity** into the model, which helps in **recognizing complex patterns**.
- Without activation functions, a machine learning model (especially neural-like models) would behave like a **linear regression model**, no matter how many layers it has.
- They help the system **decide** whether a neuron (or unit) should be **activated or not**.
- **Control output:** It controls the **range** of the output (like 0 to 1, -1 to 1, etc.).
- **Decision making:** It helps a model decide the class or value based on processed inputs.

Types of Activation records

1. **Step Function**
(Heaviside Function) $f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$

2. **Sign Function** $f(x) = \begin{cases} +1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$

3. **Sigmoid Function** $f(x) = \frac{1}{1 + e^{-x}}$
Range: $(0, 1)$

4. **Tanh Function**
(Hyperbolic Tangent) $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Range: $(-1, 1)$

Support Vector Machine

SVM is a supervised learning algorithm used for classification and sometimes regression. It is based on the idea of finding the best decision boundary (hyperplane) that segregate n-dimensional space into classes of data with the maximum margin. **Core Concepts of SVM:**

1. Hyperplane:

- The best decision surface that separates different classes.
- In 2D, it's a line; in 3D, a plane; in higher dimensions, a hyperplane.
- Goal: Find the **optimal hyperplane** that divides data **with the largest possible margin**.

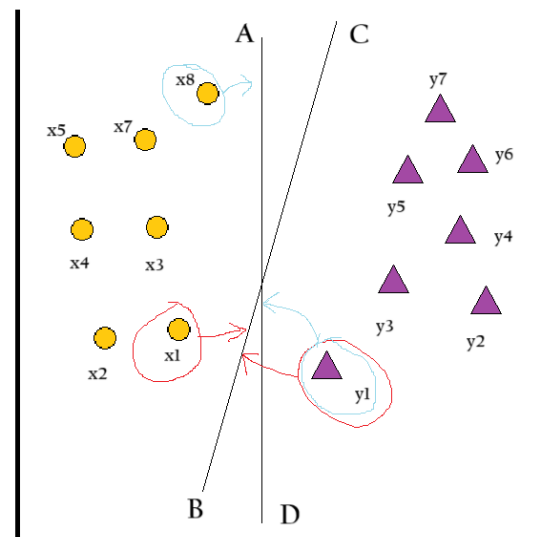
2. Margin:

- Distance between the hyperplane and the nearest data points from both classes.
- SVM aims to maximize this margin.

$$\text{Margin width} = \frac{2}{\|w\|}$$

3. Support Vectors:

- The **closest points** to the hyperplane from either class.
- They are **critical** — they define the margin and the hyperplane position.
- Removing them **changes** the hyperplane.
- Example:**
 - x1 and y1 are SV of line AB
 - x8 and y1 are SV of line CD



Objective of SVM:

- Maximize margin: $\min \frac{1}{2} \|w\|^2$
- Subject to constraints: $y_i(w^T x_i + b) \geq 1$
For all training samples x_i , with class label $y_i \in \{-1, +1\}$

Advantages

- Works well for **high-dimensional data**.
- Effective with **clear margin of separation**.
- Uses only **support vectors**, making it memory-efficient.
- Can be adapted to **non-linear** problems using kernels.
- **Versatile with Kernels**: Can model complex relationships using kernel functions.

Disadvantages

- Not ideal for **very large datasets** (high training time).
- Struggles with **noisy and overlapping classes**.
- Choosing the **right kernel and parameters** can be tricky.
- **Hard to Interpret**: Not as intuitive as decision trees or linear regression.
- **Binary Nature**: Naturally supports only binary classification (multi-class needs extensions).

Types of SVM

1) **Linear SVM**: is used when the data is linearly separable, i.e., it can be separated by a straight line (2D) or hyperplane (higher dimensions).

- Separates data using a single straight hyperplane.
- Finds the maximum margin hyperplane between the two classes.
- Faster to compute.
- Simpler mathematically and visually.

Mathematical Form:

$$f(x) = \text{sign}(w^T x + b)$$

- w = weight vector
- b = bias
- x = input features

2) **Non-Linear SVM**: is used when the data is **not linearly separable** — the classes are mixed or have curves or complex boundaries.

- Uses **kernel functions** to map data into a higher-dimensional space.
- In that space, a **linear hyperplane** can be applied (to add more dimension to the data)
- Handles complex classification problems.
- More flexible, but computationally more expensive.
- **Common Kernels**:
 - Polynomial Kernel
 - Gaussian Kernel
 - Sigmoid Kernel

Example of Linear SVM

Suppose we are given the following positively labeled data points,

$$\left\{ \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \begin{pmatrix} 6 \\ -1 \end{pmatrix} \right\}$$

and the following negatively labeled data points,

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right\}$$

- Plot the points.
- Find the support vectors
- Find weighted vector and bias
- Find equation of hyperplane

Question:

You are given the following 2D data points with class labels:

Sample	x_1	x_2	Class
A	2	2	+1
B	4	4	+1
C	4	0	-1
D	0	0	-1

- a) State the support vectors (verify them later)
- b) State hyperplane equation
- c) Predict the class of point (3,1)
- d) Find margin width